Daily Quant Digest: 2026-10-17

Today's topics

Track	Topic	Difficulty
Probability & Statistics	Order statistics and uniform spacings: Beta laws, the exponential (Renyi) representation; expected shortest, middle and longest piece of a randomly broken stick	Medium
Derivatives	Put-call parity and no-arbitrage bounds: conversions, reversals, boxes, FX parity, implied borrow, convexity in strike	Medium
Algorithms & DS	Trees: lowest common ancestor and node distances with binary lifting	Medium-Hard

1. Probability & Statistics: Order Statistics and Uniform Spacings

Order statistics show up everywhere in quant interviews: "expected max of n uniforms", "break a stick", "first of n orders to arrive", "probability the best bid comes from the third market maker". Two pictures solve almost all of them: the **Beta picture** for a single order statistic, and the **spacings picture** (spacings are exchangeable, and they are normalised exponentials).

1.1 Definitions and the single order statistic

Let $X_1, \dots, X_n$ be i.i.d. with CDF F and density f. Sort them: $X_{(1)} \le X_{(2)} \le \cdots \le X_{(n)}$. $X_{(k)}$ is the **k-th order statistic**.

CDF by counting. $X_{(k)} \le x$ iff at least k of the X_i are $\le x$. The number below x is $\mathrm{Bin}(n, F(x))$, so

$$P(X_{(k)} \le x) = \sum_{j=k}^{n} \binom{n}{j} F(x)^j (1 - F(x))^{n-j}.$$

Density by the "one in the box" heuristic. For $X_{(k)}$ to land in $[x, x+dx]$ we need exactly one point in the box, $k-1$ below and $n-k$ above. The multinomial count of ways to assign the roles is $\frac{n!}{(k-1)!\,1!\,(n-k)!}$, so

$$f_{(k)}(x) = \frac{n!}{(k-1)!\,(n-k)!}\,F(x)^{k-1}\,(1-F(x))^{n-k}\,f(x).$$

For uniforms on $[0,1]$ this is exactly the $\mathrm{Beta}(k,\,n-k+1)$ density:

$$U_{(k)} \sim \mathrm{Beta}(k, n-k+1), \qquad \mathbb{E}[U_{(k)}] = \frac{k}{n+1}, \qquad \mathrm{Var}(U_{(k)}) = \frac{k(n-k+1)}{(n+1)^2(n+2)}.$$

The mean $k/(n+1)$ is the first thing to memorise: n points cut $[0,1]$ into $n+1$ pieces, and on average each piece has length $1/(n+1)$.

Reduction to uniforms. Since $F(X_i)$ is uniform and F is monotone, $F(X_{(k)}) \overset{d}{=} U_{(k)}$. So any order-statistic question about a continuous distribution is a uniform question in disguise (quantile transform).

Joint density. By the same box heuristic, the whole sorted vector of n uniforms has density $n!$ on the simplex $\{0 < u_1 < \cdots < u_n < 1\}$. All the sorting does is fold the $n!$ orderings onto one.

1.2 Spacings are exchangeable

Define the **spacings** $D_1 = U_{(1)}$, $D_j = U_{(j)} - U_{(j-1)}$ for $2 \le j \le n$, and $D_{n+1} = 1 - U_{(n)}$. They sum to 1.

Theorem. $(D_1, \dots, D_{n+1})$ is uniformly distributed on the simplex $\{d_j \ge 0, \sum d_j = 1\}$, i.e. $\mathrm{Dirichlet}(1, \dots, 1)$. In particular the spacings are **exchangeable**: any permutation has the same joint law.

Proof sketch. The map $(u_1, \dots, u_n) \mapsto (d_1, \dots, d_n)$ is linear with determinant 1, so the density stays the constant $n!$ on the image, which is $\{d_j \ge 0, d_1 + \cdots + d_n \le 1\}$. A constant density on the simplex is symmetric in all $n+1$ coordinates (with $d_{n+1} = 1 - \sum$). $\square$

A cleaner way to see the symmetry: put $n+1$ i.i.d. uniform points on a **circle** of circumference 1 and cut at the first one. The $n+1$ arcs are obviously exchangeable by rotational symmetry, and cutting at one point turns the circle into the interval with n points.

Consequences you can use immediately:

- Every spacing has the same law as $D_1 = U_{(1)} \sim \mathrm{Beta}(1, n)$, so $P(D_j > x) = (1-x)^n$ and $\mathbb{E}[D_j] = 1/(n+1)$.
- $P(D_i > a, D_j > b) = (1-a-b)_+^n$ for $i \ne j$ (shrink the stick by $a+b$ and spread the rest).

- More generally $P(D_{i_1} > a_1, \dots, D_{i_r} > a_r) = (1 - a_1 - \cdots - a_r)_+^n$.

1.3 The exponential representation (the key trick)

Theorem. Let $E_1, \dots, E_{n+1}$ be i.i.d. $\mathrm{Exp}(1)$ and $S = E_1 + \cdots + E_{n+1}$. Then

$$(D_1, \dots, D_{n+1}) \overset{d}{=} \left(\frac{E_1}{S}, \dots, \frac{E_{n+1}}{S} \right),$$

and the normalised vector is **independent** of S (which is $\mathrm{Gamma}(n+1, 1)$).

Proof sketch. The joint density of $(E_1, \dots, E_{n+1})$ is $e^{-\sum e_j}$, which depends only on the sum. Change variables to $(s, d_1, \dots, d_n)$ with $e_j = s\, d_j$: the Jacobian is s^n, so the density factorises as $s^n e^{-s} \times$ const on the simplex. The first factor is the Gamma density, the second is uniform on the simplex. Independence falls out of the factorisation. $\square$

Intuition: Poisson process arrivals. Condition a rate-1 Poisson process to have its $(n+1)$-th arrival at time S; the first n arrivals are then i.i.d. uniform on $[0, S]$. That is the link between exponential gaps and uniform order statistics.

Renyi's representation for exponential order statistics. If $E_1, \dots, E_m$ are i.i.d. $\mathrm{Exp}(1)$, the minimum is $\mathrm{Exp}(m)$ (an exponential race among m clocks), and by memorylessness the remaining $m-1$ clocks restart afresh. Repeating:

$$E_{(k)} \overset{d}{=} \sum_{j=1}^{k} \frac{E'_j}{m - j + 1}, \qquad \mathbb{E}[E_{(k)}] = \sum_{j=1}^{k} \frac{1}{m - j + 1},$$

with E'_j i.i.d. $\mathrm{Exp}(1)$. In particular $\mathbb{E}[\max] = H_m = 1 + \tfrac{1}{2} + \cdots + \tfrac{1}{m}$.

Combining the two theorems gives the **sorted spacings**. Write $m = n+1$ for the number of pieces and $D_{[1]} \le \cdots \le D_{[m]}$ for the spacings sorted by size. Because the direction vector E/S is independent of S, and sorting commutes with dividing by S,

$$\mathbb{E}[E_{(k)}] = \mathbb{E}[S \cdot D_{[k]}] = \mathbb{E}[S]\, \mathbb{E}[D_{[k]}] = m\, \mathbb{E}[D_{[k]}],$$

so

$$\boxed{\mathbb{E}[D_{[k]}] = \frac{1}{m} \sum_{j=1}^{k} \frac{1}{m - j + 1}}$$

The shortest piece has mean $1/m^2$, the longest has mean $H_m/m \approx (\ln m)/m$.

1.4 Asymptotics and extremes

- **Max of n uniforms:** $n(1-U_{(n)}) \Rightarrow \mathrm{Exp}(1)$, since $P(n(1-U_{(n)}) > t) = (1 - t/n)^n \to e^{-t}$. Extremes of bounded variables live on the scale $1/n$.
- **Central order statistics:** for $k/n \to p$, $\sqrt{n}\,(X_{(k)} - \xi_p) \Rightarrow N\left(0, \frac{p(1-p)}{f(\xi_p)^2}\right)$ where ξ_p is the p-quantile. This is why the sample median has variance $\frac{1}{4nf(m)^2}$: for a normal it is $\frac{\pi}{2} \cdot \frac{\sigma^2}{n}$, about 57% worse than the mean, but for fat tails it can be far better.
- **Longest spacing:** $m\,D_{[m]} - \ln m$ converges to a Gumbel law. The longest gap is about $\ln m$ times the average gap. In practice: in a stream of Poisson order arrivals, the longest quiet period is much longer than intuition suggests.

1.5 Pitfalls

1. **The order statistics are dependent.** $U_{(1)}$ and $U_{(n)}$ are positively correlated ($\mathrm{Cov} = \frac{1}{(n+1)^2(n+2)}$). You cannot multiply marginals.
2. **Spacings are exchangeable but not independent.** They are negatively correlated (they sum to 1). The exponential trick restores independence, at the price of the random normaliser S.
3. **Pieces on a stick vs the piece containing a point.** The piece that contains a given point (say 0.5) is size-biased: its expected length is close to $2/(n+1)$, not $1/(n+1)$. This is the inspection paradox. Read the question carefully.
4. **Discrete distributions have ties.** The density formula assumes continuity. For dice, work with $P(X_{(k)} \le x)$ via the binomial count, never with densities.
5. **Ratio of expectations.** $\mathbb{E}[E_{(k)}/S] = \mathbb{E}[E_{(k)}]/\mathbb{E}[S]$ only because of the independence theorem. In general $\mathbb{E}[X/Y] \neq \mathbb{E}[X]/\mathbb{E}[Y]$.

1.6 Interview question

> A stick of length 1 is broken at two points chosen independently and uniformly at random.
> (a) What is the expected length of the **longest** piece?
> (b) Of the **shortest** and the **middle** piece?
> (c) Follow-up: what is the probability the three pieces form a triangle?
> (d) Follow-up: if the stick is broken at n points, what is the expected length of the longest piece?

1.7 Solution

Setup. Three pieces (D_1, D_2, D_3), uniform on the simplex $\{d_j \ge 0, d_1 + d_2 + d_3 = 1\}$ (Section 1.2). Each has mean $1/3$; we want the sorted means.

(a) Longest piece, from first principles (inclusion-exclusion).

Write $M = \max(D_1, D_2, D_3)$ and use $\mathbb{E}[M] = \int_0^1 P(M > x)\,dx$. Let $A_j = \{D_j > x\}$. Then

$$P(M > x) = P(A_1 \cup A_2 \cup A_3) = \sum_j P(A_j) - \sum_{i<j} P(A_i A_j) + P(A_1 A_2 A_3).$$

From Section 1.2 with $n = 2$ cut points: $P(A_j) = (1-x)^2$, $P(A_i A_j) = (1-2x)_+^2$, $P(A_1 A_2 A_3) = (1-3x)_+^2$. So

$$P(M > x) = 3(1-x)^2 - 3(1-2x)_+^2 + (1-3x)_+^2.$$

Integrate each term over $[0,1]$, remembering the positive parts vanish beyond $1/2$ and $1/3$:

$$\int_0^1 3(1-x)^2 dx = 1, \quad \int_0^{1/2} 3(1-2x)^2 dx = \tfrac{1}{2}, \quad \int_0^{1/3} (1-3x)^2 dx = \tfrac{1}{9}.$$

Therefore

$$\mathbb{E}[M] = 1 - \tfrac{1}{2} + \tfrac{1}{9} = \frac{11}{18} \approx 0.6111.$$

Sanity check on the formula for $P(M > x)$: at $x = 1/3$ it gives $3 \cdot \frac{4}{9} - 3 \cdot \frac{1}{9} + 0 = 1$, correct because the largest of three pieces summing to 1 is always at least $1/3$. At $x = 1/2$ it gives $3 \cdot \frac{1}{4} = \frac{3}{4}$, which is the complement of part (c).

(b) Shortest and middle. The shortest is the easiest: $\{\min > x\} = A_1 A_2 A_3$, so

$$\mathbb{E}[\min] = \int_0^{1/3} (1-3x)^2\,dx = \frac{1}{9}.$$

The middle piece follows from the sum: $\mathbb{E}[\text{mid}] = 1 - \frac{11}{18} - \frac{2}{18} = \frac{5}{18}$.

Interview trick (the exponential representation). With $m = 3$ pieces, the boxed formula from Section 1.3 gives directly

$$\mathbb{E}[D_{[1]}] = \tfrac{1}{3} \cdot \tfrac{1}{3} = \tfrac{1}{9}, \quad \mathbb{E}[D_{[2]}] = \tfrac{1}{3}\left(\tfrac{1}{3} + \tfrac{1}{2}\right) = \tfrac{5}{18}, \quad \mathbb{E}[D_{[3]}] = \tfrac{1}{3}\left(\tfrac{1}{3} + \tfrac{1}{2} + 1\right) = \tfrac{11}{18}.$$

The story to tell the interviewer: “Replace the three pieces by three independent exponential clocks and normalise. The first clock rings after $\text{Exp}(3)$, mean $1/3$; then two remain, the next gap has mean $1/2$; the last gap has mean 1. The max of three exponentials has mean $1/3 + 1/2 + 1 = 11/6$, and dividing by the expected total, 3, gives $11/18$.”

(c) Triangle. Three lengths form a triangle iff each is less than the sum of the other two, i.e. every piece is $< 1/2$. So

$$P(\text{triangle}) = 1 - P(M > \tfrac{1}{2}) = 1 - 3\left(\tfrac{1}{2}\right)^2 = \frac{1}{4}.$$

(The events $\{D_j > 1/2\}$ are disjoint since two pieces cannot both exceed half the stick, so the union is a plain sum.)

(d) n cut points. Now $m = n + 1$ pieces and

$$\mathbb{E}[\text{longest}] = \frac{H_{n+1}}{n+1}, \qquad \mathbb{E}[\text{shortest}] = \frac{1}{(n+1)^2}.$$

For $n = 10$: the average piece is $1/11 \approx 0.091$, the longest averages $H_{11}/11 \approx 0.2745$ (three times the average), the shortest $1/121 \approx 0.0083$ (eleven times smaller). The ratio longest/average is $H_m \approx \ln m + 0.577$, which grows without bound.

Sanity checks.
- The three expectations sum to 1: $\frac{2+5+11}{18} = 1$.
- Bounds: the max is in $[1/3, 1]$ and $11/18 \approx 0.611$ is in range; the min is in $[0, 1/3]$ and $1/9$ is in range.
- Monte Carlo with 10^6 samples gave 0.1110, 0.2779, 0.6111 and a triangle frequency of 0.2498, matching $1/9$, $5/18$, $11/18$, $1/4$.

Why a trader cares. Replace “stick” by “time interval” and “cut points” by Poisson order arrivals: the longest gap between trades, the time until the first of n market makers responds (Exp with n times the rate), and the “who arrives last” logic all use exactly this machinery.

---

2. Derivatives: Put-Call Parity and No-Arbitrage Bounds

Put-call parity is the one option relationship that needs **no model**: no volatility, no distribution, no Black-Scholes. It comes from a static portfolio held to expiry. Every options desk monitors it constantly, because a violation is either free money or, much more often, a sign that something you have not priced (borrow, dividends, funding, early exercise) is going on.

2.1 The identity

For European options on the same underlying, strike K, expiry T:

$$\max(S_T - K, 0) - \max(K - S_T, 0) = S_T - K \quad \text{for every } S_T.$$

Long call plus short put has exactly the payoff of a long forward struck at K. Two portfolios with identical payoffs in every state must have the same price today (otherwise

buy the cheap one, sell the rich one, lock in the difference). So

$$C - P = \mathrm{PV}(S_T) - \mathrm{PV}(K) = S_0 e^{-qT} - K e^{-rT} = e^{-rT}(F - K),$$

where q is the dividend yield (or borrow cost, or foreign rate) and $F = S_0 e^{(r-q)T}$ is the forward. The last form is the one to remember: **call minus put is the discounted value of being long the forward at K.**

Immediate consequences:
- At $K = F$ (ATM forward), $C = P$. This is why FX and rates desks quote "ATM" relative to the forward, not the spot.
- Any model must satisfy parity, so the implied vols of a call and put with the same strike and expiry must be equal. If they are not, the market is telling you the forward (or the borrow, or the dividend) you used is wrong.

2.2 Who trades it: conversions, reversals, boxes

- **Conversion:** long stock, long put, short call (same K, T). Payoff at expiry is exactly K: a synthetic zero-coupon bond. A **reversal** (reverse conversion) is the opposite: short stock, short put, long call, which borrows at an implied rate. Market makers and arb desks trade these to lock in parity mispricing, and also to manage pin risk and dividend risk.
- **Synthetic forward/futures:** long call plus short put. Used when futures are illiquid or when a desk wants to move delta without trading the underlying.
- **Box spread:** a long call spread plus a long put spread with the same strikes $K_1 < K_2$ (equivalently, a synthetic long forward at K_1 plus a synthetic short forward at K_2). Payoff is always $K_2 - K_1$. A box is a zero-coupon bond built from options, so its price reveals the market's financing rate. European-style SPX boxes are actively traded as a funding and lending instrument, priced as a spread to SOFR or T-bills.
- **FX parity:** for an option to buy one unit of foreign currency (e.g. USD call/JPY put in USDJPY), with domestic (JPY) discount factor DF_d and forward F:

$$C - P = DF_d\,(F - K) \quad \text{(in domestic per unit foreign)}.$$

  Since FX forwards come from covered interest parity, $F = S \cdot \frac{DF_f}{DF_d}$, this is the same statement as the equity one with the foreign rate playing the role of the dividend yield.

2.3 Model-free bounds

All of these come from "if payoff A $\geq$ payoff B in every state, then price A $\geq$ price B". Write $C(K)$, $P(K)$ for European prices.

Bound	Reason
$\max(e^{-rT}(F-K),0) \le C \le S_0 e^{-qT}$	the call dominates the forward and zero; the share (with its dividends) dominates the call
$\max(e^{-rT}(K-F),0) \le P \le K e^{-rT}$	the put pays at most K
$C(K)$ decreasing, $P(K)$ increasing in K	payoffs are monotone in K
$C(K_1) - C(K_2) \le e^{-rT}(K_2 - K_1)$ for $K_1 < K_2$	the call spread pays at most $K_2 - K_1$
$C(K)$ convex in K: $C(K-h) - 2C(K) + C(K+h) \ge 0$	the butterfly payoff is non-negative

The last two give the slope and curvature of the call price in strike: $-e^{-rT} \le \partial C/\partial K \le 0$ and $\partial^2 C/\partial K^2 \ge 0$. In fact $\partial^2 C/\partial K^2 = e^{-rT}\,\phi_T(K)$, the risk-neutral density (Breeden-Litzenberger). So **a negative butterfly means a negative probability**, and any vol surface you build must avoid it. This is the main constraint when fitting smiles: wild wings in implied vol space can produce negative butterflies.

American options. Parity becomes a pair of inequalities. With no dividends:

$$S_0 - K \le C_A - P_A \le S_0 - K e^{-rT}.$$

And an American call on a non-dividend-paying stock is never exercised early, so $C_A = C_E$. Reason: $C_A \ge C_E \ge S_0 - K e^{-rT} > S_0 - K$, so the option is always worth more alive than exercised. You give up the time value of the put embedded in the call, plus the interest on K. American puts, by contrast, can be optimally exercised early when deep in the money (you want to receive K now and earn interest on it).

2.4 Worked scenario 1: a parity break in USDJPY

Market data (illustrative, 3 months, simple money-market rates on ACT/360 simplified to $T = 0.25$):
- USDJPY spot $S = 150.00$
- USD rate 4.00%, JPY rate 0.50%
- Forward: $F = 150.00 \times \frac{1+0.005\times 0.25}{1+0.04\times 0.25} = 150.00 \times \frac{1.00125}{1.01} = 148.7005$ (forward points -129.95 pips; USD trades at a forward discount because USD rates are higher).
- JPY discount factor: $DF_d = 1/1.00125 = 0.998752$.

Fair value with a 10% vol for the 150.00 strike (Garman-Kohlhagen, quoted in JPY per USD): call 2.3711, put 3.6689. Parity says

$$C - P = 0.998752 \times (148.7005 - 150.00) = -1.2979.$$

The model numbers agree, as they must for any vol.

Now suppose a broker shows mids for USD 10M of the 150.00 strike: **call 2.35, put 3.80**. Then $C - P = -1.45$, which is 0.152 JPY per USD too low: the put is rich relative to the call.

The trade (a reversal in FX form):
1. Sell the put at 3.80 and buy the call at 2.35: receive 1.45 JPY per USD today, i.e. JPY 14.5M on USD 10M. This is a synthetic long USD forward at 150.00.
2. Hedge by selling USD 10M 3M outright forward at 148.7005.
3. Deposit the JPY 14.5M for 3 months at 0.50%: it grows to $14{,}500{,}000 = $ JPY 14,518,125.

At expiry, whatever the spot: the option pair makes you buy USD 10M at 150.00 (if spot is above 150 you exercise the call; if below, the put is exercised against you). You deliver it into the forward at 148.7005. Loss on that leg: $(150.00 - 148.7005) = $ JPY 12,995,050.

Net: $14{,}518{,}125 - 12{,}995{,}050 = $ **JPY 1,523,075** (about USD 10,240), locked in regardless of where USDJPY goes. In PV terms: $0.1521 \times 10\text{M}$ discounted, matching the mispricing.

Reality check. This is roughly 1.5 pips of premium on a liquid pair; in practice you would pay bid-offer on two options and a forward, which would eat most of it. Real parity breaks in the interbank market are tiny and short-lived. A persistent apparent break usually means your forward is wrong (the market is using a different basis-adjusted forward, e.g. cross-currency basis), not that there is free money.

2.5 Worked scenario 2: implied borrow on a hard-to-borrow stock

Stock at $S = 50$, $r = 4\%$, $T = 0.5$, no dividends. The 50-strike European-style options trade at $C = 4.10$, $P = 5.60$.

Parity with zero borrow cost says $C - P = 50 - 50e^{-0.02} = 50 - 49.0099 = 0.9901$. The market has $C - P = -1.50$: the put is 2.49 too expensive relative to the call. Is it an arbitrage? The "trade" would be: sell the put, buy the call, short the stock. But you cannot short the stock cheaply: it is hard to borrow, and the lender charges a fee.

Solve for the borrow cost q that restores parity:

$$Se^{-qT} = C - P + Ke^{-rT} = -1.50 + 49.0099 = 47.5099 \quad \Rightarrow \quad q = -\frac{1}{0.5}\ln\frac{47.5099}{50} = 10$$

The options market is pricing a 10.2% annual borrow fee. If your prime broker can locate the stock at 3%, the reversal is profitable; if the borrow is 15% or gets recalled, it is not. Parity has turned into a **borrow rate quote**. (This is exactly what happened in meme-stock squeezes: puts traded far above parity-with-zero-borrow because borrow was scarce.)

2.6 Worked scenario 3: a convexity violation

European calls, same expiry: $C(95) = 7.00$, $C(100) = 4.20$, $C(105) = 1.30$.

Butterfly: $C(95) - 2C(100) + C(105) = 7.00 - 8.40 + 1.30 = -0.10$.

Buy the 95/100/105 butterfly (buy one 95, sell two 100, buy one 105): you **receive** 0.10 today, and the payoff at expiry is ≥ 0 in every state (a tent peaking at 5 when $S_T = 100$). Free money plus a free option. In a vol surface fit, this shows up as the implied vol at 100 being too high relative to its neighbours, enough to create a negative implied density.

2.7 What can go wrong

- **American vs European.** Parity is an equality only for Europeans. With American single-stock options, early exercise breaks the static hedge. Famous retail blowup (2019): a trader sold American-style SPY box spreads believing they were risk-free borrowing; the short legs were assigned early and the account was wiped out. SPX boxes are European, SPY boxes are not.
- **Dividends.** An unexpected special dividend changes q and the forward; conversions held into the ex-date carry dividend risk. Short calls on dividend payers can be exercised early the day before ex-date (dividend arbitrage).
- **Pin risk.** At expiry with $S_T \approx K$, you do not know whether your short option will be assigned, so you do not know your stock position over the weekend. Conversions and reversals are exactly the positions exposed to this.
- **Funding and borrow.** The "r" and "q" in parity are your actual funding and borrow rates, not textbook rates. Different desks see different arbitrage bands.
- **Settlement mismatches.** Cash-settled index options vs physically settled ETF options vs futures options: the "same" parity can involve different settlement dates and discounting.

2.8 Likely interview questions

1. **"A 1-year ATM call and put on a non-dividend stock: which is more expensive?"** If ATM means strike = spot, the call: $C - P = S(1 - e^{-rT}) > 0$ for $r > 0$. If ATM means strike = forward, they are equal. Always ask which ATM.
2. **"The 100 call is 5 and the 105 call is 6. What do you do?"** Calls must be decreasing in strike. Buy the 100, sell the 105: you receive 1 today and the call spread payoff is non-negative. Total risk-free profit is at least 1.
3. **"Why is an American call on a non-dividend stock never exercised early, but an American put sometimes is?"** The call is worth at least $S - Ke^{-rT} > S - K$ alive; exercising early pays K sooner (bad) and throws away the downside protection. For a deep ITM put, exercising receives K now; the interest on K can exceed the remaining optionality, so early exercise can be optimal.

---

3. Algorithms & Data Structures: Node Distances via Lowest Common Ancestor

3.1 Problem

A binary tree has n nodes labelled $0, \dots, n-1$, rooted at 0, described by arrays `left[i]` and `right[i]` (`-1` if the child is missing). Answer Q queries: for each pair (u, v), return the number of edges on the path between u and v.

Constraints: $1 \le n \le 10^5$, $1 \le Q \le 10^5$. The tree can be arbitrarily unbalanced (depth up to $n-1$).

Example.

```
        0
       / \
      1   2
     / \   \
    3   4   5
       / \
      6   7
```

`left = [1,3,-1,-1,6,-1,-1,-1]`, `right = [2,4,5,-1,7,-1,-1,-1]`

Query	LCA	Distance
(6, 7)	4	2
(3, 7)	1	3
(6, 5)	0	5
(4, 4)	4	0
(1, 6)	1	2

3.2 From first principles

The key identity. In a tree there is exactly one path between two nodes, and it goes up from u to their **lowest common ancestor** (the deepest node that is an ancestor of both) and then down to v. With depth measured from the root,

$$\operatorname{dist}(u, v) = \operatorname{depth}(u) + \operatorname{depth}(v) - 2\operatorname{depth}(\operatorname{lca}(u, v)).$$

So the problem is really "answer Q LCA queries fast".

Single query (the classic interview version). Recursive: `lca(node)` returns `node` if it is u or v; otherwise recurse left and right; if both sides return non-null, `node` is the LCA; else pass up whichever is non-null. $O(n)$ per query. For a BST you can do better: walk down

from the root, going left if both values are smaller, right if both are larger, and stop at the split, $O(h)$.

Brute force for many queries. Store parent pointers and depths with one traversal. For each query, repeatedly move the deeper node to its parent until the two meet. Correct, but $O(h)$ per query; on a path-shaped tree that is $O(n)$, so $O(nQ) = 10^{10}$ in the worst case. Too slow.

The insight: jump in powers of two. Walking up one step at a time is like counting in unary. Precompute, for every node v and every j, the 2^j-th ancestor $\mathrm{up}[v][j]$. Then:
- $\mathrm{up}[v][0]$ is the parent.
- $\mathrm{up}[v][j] = \mathrm{up}[\mathrm{up}[v][j-1]][j-1]$: two jumps of 2^{j-1} make a jump of 2^j. That is a tiny dynamic programme, filled level by level.

Query in two phases.
1. **Equalise depths.** If u is deeper by d, write d in binary and jump 2^j for every set bit j. At most $\log_2 n$ jumps.
2. **Climb together.** If $u = v$ now, done (v was an ancestor of u). Otherwise, for j from large to small: if $\mathrm{up}[u][j] \neq \mathrm{up}[v][j]$, jump both. Jumping when the ancestors differ is always safe (we stay strictly below the LCA), and skipping when they agree avoids overshooting. Going from large j to small builds the distance to "just below the LCA" greedily, like reading off binary digits. At the end, u and v are distinct children of the LCA, which is $\mathrm{up}[u][0]$.

The ancestors of the root are set to the root itself, so jumps that would go "above the root" simply stay there. That sentinel removes all bounds checks.

Complexity. Preprocessing $O(n \log n)$ time and memory (17 entries per node for $n \le 10^5$, about 6.8 MB of `int`s). Each query $O(\log n)$. Total $O((n+Q)\log n)$, a few million operations.

Alternatives worth naming in an interview.
- **Euler tour + RMQ:** record nodes during a DFS; the LCA of u, v is the shallowest node between their first occurrences. With a sparse table, $O(n \log n)$ preprocessing and $O(1)$ query.
- **Tarjan's offline LCA:** if all queries are known in advance, one DFS with union-find answers all of them in near-linear time (union-find is coming later in the curriculum).
- Binary lifting has an extra advantage: it also answers "k-th ancestor of v" in $O(\log n)$, which the others do not do directly.

3.3 C++ solution

```cpp
// Daily Quant Digest 2026-10-17
// Distance between two nodes in a binary tree, many queries.
// Lowest common ancestor via binary lifting: O(n log n) preprocessing, O(log n) per query.
// Compile: g++ -std=c++17 -O2 -Wall 2026-10-17.cpp -o lca && ./lca

#include <array>
#include <cassert>
```

```cpp
#include <iostream>
#include <queue>
#include <random>
#include <utility>
#include <vector>

class TreeLCA {
public:
    static constexpr int LOG = 17;  // 2^17 = 131072 > 1e5 nodes

    // left[i], right[i] are the children of node i (-1 if missing). Root is `root`.
    TreeLCA(const std::vector<int>& left, const std::vector<int>& right, int root = 0)
        : n_(static_cast<int>(left.size())), depth_(n_, 0), up_(n_) {
        // BFS from the root: sets depth and the direct parent (up_[v][0]).
        // Iterative on purpose: a skewed tree of 1e5 nodes would overflow a recursive DFS.
        std::queue<int> q;
        q.push(root);
        up_[root].fill(root);  // the root's ancestors are all the root itself
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int c : {left[v], right[v]}) {  // range-for over an initializer_list
                if (c == -1) continue;
                depth_[c] = depth_[v] + 1;
                up_[c][0] = v;
                q.push(c);
            }
        }
        // up_[v][j] = 2^j-th ancestor = 2^(j-1)-th ancestor of the 2^(j-1)-th ancestor.
        // Loop j outside so that level j-1 is complete for every node before level j.
        for (int j = 1; j < LOG; ++j)
            for (int v = 0; v < n_; ++v)
                up_[v][j] = up_[up_[v][j - 1]][j - 1];
    }

    int lca(int u, int v) const {
        if (depth_[u] < depth_[v]) std::swap(u, v);  // make u the deeper node
        // Step 1: lift u up by the depth difference, one binary digit at a time.
        int diff = depth_[u] - depth_[v];
        for (int j = 0; diff > 0; ++j, diff >>= 1)
            if (diff & 1) u = up_[u][j];
        if (u == v) return u;  // v was an ancestor of u
        // Step 2: lift both while their ancestors differ, largest jumps first.
        for (int j = LOG - 1; j >= 0; --j) {
            if (up_[u][j] != up_[v][j]) {
                u = up_[u][j];
                v = up_[v][j];
            }
        }
        return up_[u][0];  // now u and v are children of the LCA
    }

    int distance(int u, int v) const {
        return depth_[u] + depth_[v] - 2 * depth_[lca(u, v)];
    }

    int depth(int v) const { return depth_[v]; }

private:
    int n_;
    std::vector<int> depth_;
    std::vector<std::array<int, LOG>> up_;  // contiguous rows of 17 ints per node
};

// Brute force for testing: walk the deeper node up via parent pointers. O(n) per query.
int bruteDistance(const std::vector<int>& parent, const std::vector<int>& depth, int u, int v) {
```

```cpp
    int d = 0;
    while (u != v) {
        if (depth[u] >= depth[v]) u = parent[u];
        else v = parent[v];
        ++d;
    }
    return d;
}

int main() {
    /* Example tree:
              0
             / \
            1   2
           / \   \
          3   4   5
             / \
            6   7
    */
    std::vector<int> left  = {1, 3, -1, -1, 6, -1, -1, -1};
    std::vector<int> right = {2, 4,  5, -1, 7, -1, -1, -1};
    TreeLCA t(left, right);

    struct Case { int u, v, lca, dist; };
    std::vector<Case> cases = {
        {6, 7, 4, 2}, {3, 7, 1, 3}, {6, 5, 0, 5}, {4, 4, 4, 0}, {1, 6, 1, 2}, {0, 7, 0, 3}};
    for (const auto& c : cases) {
        int l = t.lca(c.u, c.v), d = t.distance(c.u, c.v);
        std::cout << "lca(" << c.u << "," << c.v << ") = " << l << ", distance = " << d << '\n';
        assert(l == c.lca && d == c.dist);
    }

    // Randomised test against the brute force, including a fully skewed tree (a path).
    std::mt19937 rng(17);
    for (int trial = 0; trial < 200; ++trial) {
        int n = 1 + static_cast<int>(rng() % 300);
        std::vector<int> L(n, -1), R(n, -1), parent(n, 0), dep(n, 0);
        bool path = (trial % 10 == 0);
        for (int v = 1; v < n; ++v) {
            // Attach v as a free child of a random earlier node (or of v-1 for a path).
            while (true) {
                int p = path ? v - 1 : static_cast<int>(rng() % v);
                int& slot = (rng() & 1) ? L[p] : R[p];  // reference into L or R
                if (slot == -1) { slot = v; parent[v] = p; dep[v] = dep[p] + 1; break; }
            }
        }
        TreeLCA tt(L, R);
        for (int q = 0; q < 200; ++q) {
            int u = static_cast<int>(rng() % n), v = static_cast<int>(rng() % n);
            assert(tt.distance(u, v) == bruteDistance(parent, dep, u, v));
        }
    }

    // Stress: a path of 100000 nodes (worst case depth), deepest node vs root.
    int big = 100000;
    std::vector<int> L(big, -1), R(big, -1);
    for (int v = 0; v + 1 < big; ++v) L[v] = v + 1;
    TreeLCA path(L, R);
    assert(path.distance(big - 1, 0) == big - 1);
    assert(path.lca(big - 1, 54321) == 54321);

    std::cout << "All tests passed.\n";
    return 0;
}
```

Compiled with `g++ -std=c++17 -O2 -Wall` and run: all example cases print the expected LCA and distance, 200 random trees (including path-shaped ones) agree with the brute force on 40,000 queries, and a 100,000-node path is handled without stack issues.

3.4 Walkthrough of the C++

- **`class TreeLCA` with a constructor that does all the work.** The object is built once and then answers queries with `const` member functions (`int lca(int u, int v) const`). The `const` after the parameter list promises the method does not modify the object, so you can call it on a `const TreeLCA&`. Get in the habit of marking every read-only method `const`.
- **`static constexpr int LOG = 17;`** A compile-time constant shared by all instances. `constexpr` means the value is known at compile time, which is required because it is used as a template argument in `std::array<int, LOG>`.
- **Member initialiser list.** `: n_(...), depth_(n_, 0), up_(n_)` constructs members directly instead of default-constructing and then assigning. Members are initialised in **declaration order** (here `n_`, then `depth_`, then `up_`), not in the order you write them in the list, so `depth_(n_, 0)` is only correct because `n_` is declared first. `-Wall` warns (`-Wreorder`) if the two orders differ.
- **`std::vector<std::array<int, LOG>>`.** Each node gets a fixed-size row of 17 ints stored inline, and the vector stores the rows contiguously. Compare `std::vector<std::vector<int>>`, which would make 10^5 separate heap allocations and scatter rows across memory. For tables with a fixed inner size, `std::array` is faster and simpler. `up_[root].fill(root)` uses `std::array::fill`.
- **Iterative BFS with `std::queue<int>`.** `q.front()` reads the next node and `q.pop()` removes it (`pop` returns `void`, so you must read first). Using BFS rather than recursion matters: a recursive DFS on a 100,000-deep path would use 100,000 stack frames and can crash with the default stack size.
- **`for (int c : {left[v], right[v]})`.** A range-based for loop over a braced `std::initializer_list<int>`: a neat way to treat "both children" uniformly without duplicating code.
- **Filling the table, loop order.** The outer loop is over `j` so that the entire level `j-1` is ready before any `up_[...][j]` is computed. In BFS order, swapping the loops would also work (parents are processed before children), but the level-major order is correct for any node numbering, which is a safer habit.
- **Bit tricks in phase 1.** `for (int j = 0; diff > 0; ++j, diff >>= 1) if (diff & 1) ...` walks the binary digits of `diff`: `diff & 1` tests the lowest bit, `diff >>= 1` shifts it out, and the comma operator lets the loop advance two variables at once.
- **`std::swap(u, v)`** on the function's by-value parameters, so the caller's variables are untouched.
- **References in the test generator.** `int& slot = (rng() & 1) ? L[p] : R[p];` binds a reference to whichever element the ternary selects (both branches are lvalues of the same type, so the result is an lvalue). Assigning `slot = v` then writes straight into `L` or `R`. A plain `int slot = ...` would copy and the assignment would be lost: a classic bug.

- `const auto& c : cases` iterates without copying each `Case`. `struct Case { int u, v, lca, dist; };` is an aggregate, so `{6, 7, 4, 2}` initialises it member by member.
- `std::mt19937 rng(17)` is a seeded Mersenne Twister, so the random test is reproducible. `rng() % n` has a tiny modulo bias, fine for tests but use `std::uniform_int_distribution` when the distribution matters.
- `assert` checks invariants in debug and test code. Note that `-DNDEBUG` removes all `assert`s, so never put side effects inside one.

3.5 Variations to be ready for

- **LCA in a BST:** use the ordering, $O(h)$, no extra memory.
- **LCA when u or v may be absent:** the standard recursion returns the one that exists; track a "found both" count and return null otherwise.
- **LCA with parent pointers, no root access:** this is "intersection of two linked lists". Walk two pointers; when one reaches the root jump to the other start; they meet at the LCA after at most $\text{depth}(u) + \text{depth}(v)$ steps.
- **Weighted distances:** store the distance from the root instead of the depth; the same identity holds.
- **k-th node on the path from u to v:** find the LCA, then use the lifting table to take the k-th ancestor of u or the appropriate ancestor of v.